Graph Attention in a Learned Branching Heuristic for SAT

Donato Meoli
Dipartimento di Informatica, Università di Pisa
Largo B. Pontecorvo 3, 56127 Pisa – Italy
donato.meoli@outlook.com

October 28, 2026

Abstract

We study the branching heuristic of a Conflict-Driven Clause Learning solver for the Boolean satisfiability problem when it is learned by reinforcement, as in Graph-Q-SAT, where a graph network reads the bipartite graph of the formula and is trained by Deep Q-Learning to reduce the number of decisions of the solver. We add to the core block of that network two edge-featured graph attention layers, which weight the neighbors of each node by learned coefficients before the global attribute is updated, and we compare the resulting model (GAT-Q-SAT) with Graph-Q-SAT at matched training set and configuration, together with two controls that separate the effect of the attention from that of the added parameters. On graph coloring GAT-Q-SAT takes fewer decisions than Graph-Q-SAT on all the families we test, and the difference tends to grow with the size of the instances, since Graph-Q-SAT degrades almost to the level of the reference solver on the largest ones. [One sentence on what the controls show.] Trained on uniform random 3-SAT and tested on graph coloring, GAT-Q-SAT is again the better of the two, and it also appears to be the better one on some structured families that neither model has seen in training. Yet, on satisfiable uniform random 3-SAT Graph-Q-SAT is clearly superior, and on instances of this size neither model is competitive with the solver in running time, each decision costing a forward pass of the network. This evidence rests on three training seeds per model on graph coloring (two per control), and on one run per training family elsewhere. Computational results are reported.

1 Introduction

We are concerned with the branching heuristic of a solver for the Boolean satisfiability problem (SAT), i.e., the problem of deciding whether a formula in conjunctive normal form

$$\varphi = \bigwedge_{j=1}^{m} \bigvee_{\ell \in C_j} \ell \tag{1}$$

over n Boolean variables admits an assignment that satisfies all the clauses C_j (each literal ℓ being a variable or its negation). We consider solvers that implement Conflict-Driven Clause Learning (CDCL): the search repeatedly (i) decides the value of an unassigned variable, (ii) propagates the assignments implied by the clauses, and (iii) upon a conflict learns a new clause and backjumps to the level that the clause allows [21, 20]. In practice the choice at point (i) is delegated to a heuristic, usually some variant of the Variable State Independent Decaying Sum (VSIDS) [25] (see, e.g., [18] for an alternative), which ranks the variables by activity counters that are increased at each conflict and periodically decayed. These counters usually carry little information at the start of the search, and in any case they are only a rough surrogate of what

one would like to know (say, the variable whose assignment leads to the smallest search tree). The heuristic is therefore a natural place for a learned policy, and a convenient one, since a poor decision costs time but not correctness (the solver is sound and complete whatever the policy does). We restrict ourselves to the choice of the variable and of its value, and we leave restarts, clause deletion and preprocessing as they are in MiniSat [6].

Learning for SAT has a large literature, which cannot be reviewed here; we refer to [11] for a survey and to [17] for a benchmark of graph networks on SAT problems, and only recall the works that bear on ours. NeuroSAT [28] predicts satisfiability from single-bit supervision, which shows that message passing [9] over the graph of a formula captures some of its structure. NeuroCore [27] predicts the variables that occur in unsatisfiable cores and uses the prediction to re-rank the VSIDS activities periodically, while NeuroBack [34] queries the network once, before the search, so that inference is paid offline. Other works learn heuristics for local search [35, 36]. Learning the branching rule by reinforcement goes back at least to [16], and the analogous problem for Mixed-Integer Programs has been approached with imitation learning and graph networks [14, 8] (cf. [3] for a survey of learning in combinatorial optimization). Graph-Q-SAT [15], on which we build, represents the state of the search as the bipartite graph of the formula, and learns the action-value function of the corresponding decision process with Deep Q-Learning (DQN) [23]. Since the network reads a graph, one set of weights applies to formulas of any size, and the heuristic can be evaluated on instances larger than (or different from) the training ones. This model has been extended with new graph representations and with "restricted" heuristics that query the network on part of the decisions only [29], and [24] studies how much of the reported gains depends on the experimental choices around the method. Graph attention [31, 4] appears in some of the works above but, to the best of our knowledge, not inside the core block of Graph-Q-SAT.

In this note we add to the core block of Graph-Q-SAT two graph attention layers that use the edge embeddings computed by the block, and we compare the resulting model (GAT-Q-SAT) with Graph-Q-SAT. We make the comparison at matched training set and configuration, on graph coloring (from instances of the size of the training ones to instances four times larger), on uniform random 3-SAT and, with no retraining, on thirteen structured SATLIB families that neither model has seen, together with small-world coloring at nine levels of structure. Since the attention layers are added to the block (the sum aggregation of Graph-Q-SAT being kept), GAT-Q-SAT has about 60% more parameters than Graph-Q-SAT. In order to separate the effect of the attention from that of the capacity we also train two controls, i.e., Graph-Q-SAT with wider hidden layers and the same number of parameters as GAT-Q-SAT (Graph-Q-SAT wide), and a model in which attention replaces the sum at the capacity of Graph-Q-SAT (attention aggregation). We use three training seeds for each of the two main models on graph coloring and two for each control, which is enough to see the larger differences but not always the smaller ones (i.e., those of a few tenths of the measure used below). As in [15] the results are measured in solver iterations, and the running time is reported separately. The note also documents a port of the original implementation to a current software stack (without which the experiments could not have been run), together with the test used to check that the port has not changed what the models compute. The code of both models and of the experiments is available at `https://github.com/dmeoli/GQSAT`.

The structure of the note is as follows. In section 2 we recall the decision process defined by the branching problem and the algorithm used to learn it, with the training configuration of all the runs. Section 3 describes the graph representation of the state and the network of Graph-Q-SAT, and section 4 the attention layers that GAT-Q-SAT adds to it; the port and the reproduction test are described in section 5. In section 6 we report the computational results, ordered by the distance between the training and the test instances (graph coloring at increasing size, with the two controls, a change of the training distribution, cross-domain transfer), followed by uniform random 3-SAT and by the running time. Finally, in section 7

we draw some conclusions, and the appendix describes the instances and how the numbers of section 6 are regenerated. Instance families keep their SATLIB [12] names: `uf`n-m and `uuf`n-m are satisfiable and unsatisfiable uniform random 3-SAT with n variables and m clauses, and `flat`n-m are the 3-coloring problems of graphs with n nodes and m edges, generated as in [5].

2 Branching as a decision process

Following [15], the solver is the environment of a Markov Decision Process. Its state is the formula (1) simplified by the current partial assignment (together with the clauses learned so far), and its actions are the pairs of an unassigned variable and a truth value. Transitions are computed by the solver itself (cf. [15, section 3]): a step is one decision followed by the propagation, the conflict analysis and the backjumping that it triggers, and therefore the transition is deterministic but known only by running it. A reward of -0.1 is given at each non-terminal step and 0 at termination; hence, maximizing the return means terminating in as few decisions as possible. Note that the formula is first simplified by the variable elimination of MiniSat, which is part of the environment.

The action-value function

$$Q^\star(s,a) = \mathbb{E}\big[\, r + \gamma \max_{a'} Q^\star(s',a') \,\big] \tag{2}$$

is approximated by a network Q_θ trained by DQN [23], which minimizes the squared one-step temporal-difference error on minibatches drawn uniformly from a replay buffer, with the bootstrap target computed by a copy θ^- of the weights that is refreshed periodically, and with the transitions collected by an ε-greedy policy (neither the double estimator of [30] nor the prioritized replay of [26] is used). The learned heuristic is then $\pi(s) = \arg\max_a Q_\theta(s,a)$. Two features of the problem constrain the approximator: the number of actions changes along an episode (two per unassigned variable), and the state is defined up to a permutation of the variables and of the clauses. Hence, the network must score a variable number of actions and be invariant under those permutations, which rules out fixed-size encodings such as that of [33] and leads to the graph representation of section 3.

All the runs of this note use the configuration of [15], which we list here since it is not repeated elsewhere. Training uses Adam with a constant learning rate of $2\cdot10^{-5}$, the discount is $\gamma = 0.99$, the replay buffer holds 20000 transitions and the minibatches have 64 of them. One update is made every 4 transitions after the first 5000, and θ is copied into θ^- every 10 updates. Training stops after 50000 updates (about $2 \cdot 10^5$ transitions). The exploration rate ε is 1 for the first 5000 transitions, then decreases linearly (by 0.99/30000 per transition) until transition 30000, where it is 0.175. From there on it is 0.01; this jump comes from the schedule of the released code, which compares the length of the decay with the total number of transitions. During training an episode may take at most 500 decisions of the model (the *cap*), after which the solver completes the search with VSIDS in a single step; at test time the cap is a parameter of the evaluation (cf. section 6). Also, the gradient norm is clipped at 0.1 for the runs trained on graph coloring and at 1 for those trained on uniform random 3-SAT (as in the respective configurations of [15]).

3 Graph-Q-SAT

The state is represented [15] as a bipartite graph whose nodes are the unassigned variables and the clauses (original and learned) that are not yet satisfied, each clause restricted to its literals that are not false. There is an edge between a variable and each clause in which it occurs (stored in both directions). Node features are 2-dimensional one-hot vectors that distinguish

the two kinds of node, edge features encode in the same way the polarity of the literal, and the global attribute is a constant. The network produces two Q-values on each variable node (one per truth value), and the agent takes the arg max over all of them.

As in [15], the network is an encode–process–decode graph network [2] (cf. Figure 1). An encoder maps the edges, the nodes and the global attribute independently, each by a linear layer followed by a ReLU and a layer normalization, to embeddings of size 32. Then, the core block (whose weights are shared) is applied $M = 4$ times to the concatenation of the encoded graph and of its previous output (zero at the first round). We denote by e_{ij} the embedding of the edge from node i to node j (one for each direction), by v_i that of node i, by u the global attribute, and by $\mathcal{N}_i$ the set of the neighbors of i in the bipartite graph. Each application of the block updates first the edges, then the nodes, then the global attribute:

$$e'_{ij} = \phi^e\big(e_{ij}, v_i, v_j, u\big)\,, \quad v'_i = \phi^v\Big(\textstyle\sum_{j\in\mathcal{N}_i} e'_{ij},\, v_i,\, u\Big)\,, \quad u' = \phi^u\big(\bar{e}', \bar{v}', u\big)\,, \tag{3}$$

where $\bar{e}'$ and $\bar{v}'$ are the means of the updated edge and node embeddings, and ϕ^e, ϕ^v and ϕ^u are perceptrons with one hidden layer of 64 units and outputs of size 64, each followed by a ReLU and a layer normalization. Finally, a decoder of the same form as the encoder maps the embedding of each variable node to 32 components, and a linear layer maps these to the two Q-values of the node. Since the weights are shared across the nodes and across the M rounds, one set of them applies to a formula with any number of variables and clauses. With this configuration the network has 71144 parameters. (The figures follow the notation of [2], in which V and E are the sets of the node and edge embeddings and ρ denotes an aggregation, e.g., the sum in (3).)

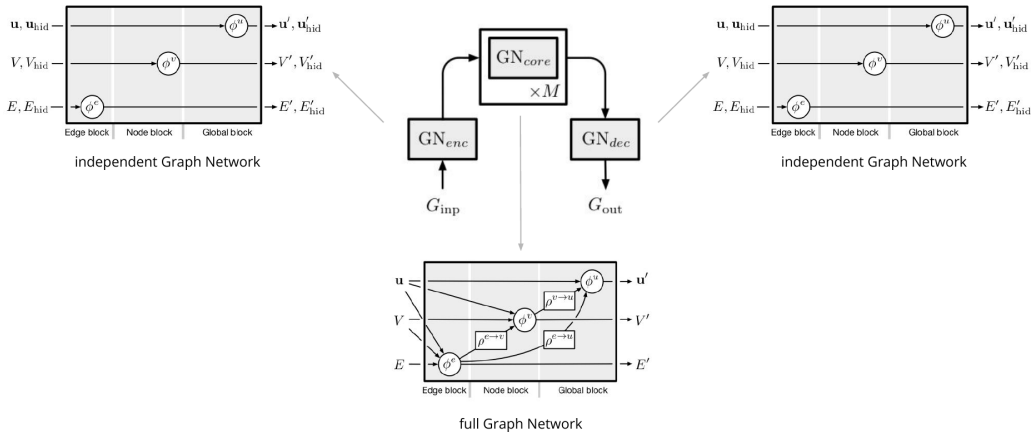


Figure 1: The encode–process–decode network of Graph-Q-SAT: an independent block encodes the edges, the nodes and the global attribute, a full graph-network block is applied M times with shared weights, and a second independent block decodes the Q-values. Blocks drawn after [2].

Note that the node update of (3) sums the messages of the edges incident to a node, and therefore it gives the same weight to all the clauses in which a variable occurs (and to all the variables of a clause). For a formula drawn uniformly at random at the phase transition [22] this seems a reasonable choice, the clauses being statistically alike. In a formula that encodes a structured problem, instead, the clauses of a variable often play different roles (in graph coloring, e.g., "at least one color" for a node and "different colors" for the two ends of an edge), and one may expect a weighting of the neighbors to be of some use.

4 GAT-Q-SAT

GAT-Q-SAT inserts two graph attention layers [31] in the core block, between the node update and the global update of (3). Thus, the node embeddings are recomputed from those of their neighbors, with learned weights, before u' is computed. Each layer maps the embeddings v_i of the nodes (the first one those computed by ϕ^v) into

$$v_i^+ = \Big\|_{k=1}^{K} \sum_{j \in \mathcal{N}_i} \alpha_{ij}^k \, W_k v_j + b \,, \qquad \alpha_{ij}^k = \frac{\exp \omega_{ij}^k}{\sum_{j' \in \mathcal{N}_i} \exp \omega_{ij'}^k} \,, \tag{4}$$
$$\omega_{ij}^k = \mathrm{LeakyReLU}\big(\mathbf{w}_k^\top [\, W_k v_i \,\|\, W_k v_j \,\|\, W_k^e e'_{ji} \,]\big) \,,$$

where K is the number of heads, $\|$ denotes concatenation, the LeakyReLU has slope 0.2 on the negative values, and e'_{ji} is the embedding of the edge from j to i computed by ϕ^e in the same application of the block (no self-loop is added, since the graph is bipartite). Hence, the coefficients α_{ij}^k depend on the current edge embeddings, which carry the polarity of the literal together with what the block has added to it. In our configuration the first layer has $K = 3$ heads of size 64 (concatenated into 192 components) and the second a single head, which returns to size 64; each layer is followed by a ReLU, and the output by a layer normalization. (In the notation of Figure 2, α^e are the coefficients α_{ij}^k and β^e the messages $W_k v_j$.) All the rest of the network (the encoder, the edge and node updates with their sum, the global update, the decoder, and M) is unchanged, and so is the training configuration of section 2. In the terms of [2], the resulting block is a full graph-network block with a non-local block inserted between the node and the global update (cf. Figure 2).

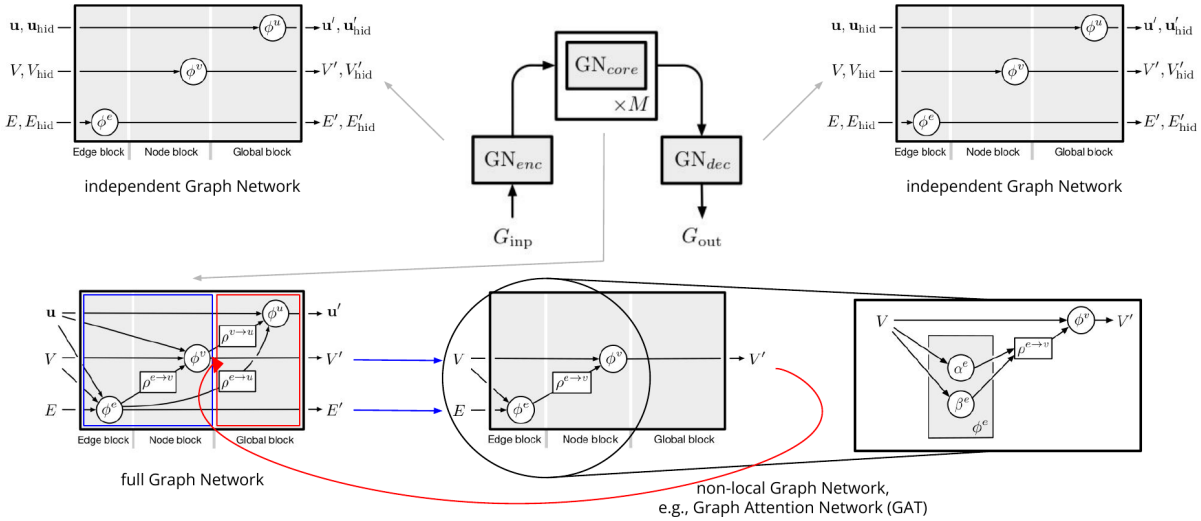


Figure 2: The core block of GAT-Q-SAT: the block of Figure 1 with a non-local block of two attention layers (4) inserted between the node and the global update. Blocks drawn after [2].

The attention layers add 42112 parameters to the 71144 of Graph-Q-SAT (about 60% more), and their cost is paid at each decision, since the coefficients are computed on all the edges at each application of the block. Of course, the coefficients can only be of use where the neighbors of a node differ in a way that matters for the decision. On a uniform random formula at the phase transition one may expect them to learn little (if anything), and the added parameters to make the fit somewhat harder.

Controls. Since the layers are added and the sum of (3) is kept, a difference between GAT-Q-SAT and Graph-Q-SAT may be due to the attention or simply to the larger network. We therefore train two further models with the same configuration. The first one (Graph-Q-SAT

wide) is Graph-Q-SAT with hidden layers of 104 units in place of 64, which gives 113504 parameters (against the 113256 of GAT-Q-SAT) and no attention. The second one (attention aggregation) keeps the block of (3) and replaces its sum by

$$\sum_{j \in \mathcal{N}_i} \bar{\lambda}_{ij}\, e'_{ij}\,, \qquad \bar{\lambda}_{ij} = \frac{1}{K} \sum_{k=1}^{K} \lambda^k_{ij}\,,$$
$$\lambda^k_{ij} = \frac{\exp \mu^k_{ij}}{\sum_{j' \in \mathcal{N}_i} \exp \mu^k_{ij'}}\,, \qquad \mu^k_{ij} = \mathrm{LeakyReLU}\big(\mathbf{z}_k^\top [\, v_i \,\|\, e'_{ij}\,]\big)\,, \tag{5}$$

with $K = 3$ heads, whose coefficients are averaged so that ϕ^v receives an input of the same size as with the sum. The vectors $\mathbf{z}_k$ (each with a bias, which the normalization cancels) add 483 parameters to the 71144 of Graph-Q-SAT. The first control has the capacity of GAT-Q-SAT without its attention, and the second one has the attention (in a simpler form) without the added capacity. Note that a weighted mean such as (5) does not carry the degree of the node, which the sum does; this may matter on graphs whose degrees vary, and it is one of the reasons why the second control is not simply a smaller GAT-Q-SAT.

5 Implementation

The original implementation of [15] is written in PyTorch 1.1 with pinned versions of PyTorch Geometric [7] and of the compiled extensions `torch-scatter` and `torch-sparse`. It drives a modified MiniSat 2.2 [6], exposed to Python as a `gym` environment through a SWIG wrapper. These versions can no longer be installed together on a current system, and therefore we ported the code to current releases of PyTorch and PyTorch Geometric. In the port the aggregations of (3) are taken from `torch_geometric.utils.scatter`, which removes the two compiled extensions. The environment is constructed directly, without `gym.make` (whose checks reject the non-standard signatures of the MiniSat wrapper), which allows the same code to run with `gym` and with `gymnasium`; the SWIG wrapper is regenerated and rebuilt against `numpy` 2. Also, the version of PyTorch Geometric used in 2021 stores the linear map of an attention layer twice (`lin_src` and `lin_dst`), where the current one has a single `lin`. When a checkpoint of 2021 is loaded one of the two copies is kept; this loses nothing, since in all our checkpoints the two are equal.

In order to check that the port has not changed what the models compute, we re-ran on the new stack the evaluation of the checkpoints trained in 2021 on the original one, and compared the logs instance by instance. In 63 of the 86 repeated evaluations the number of iterations of the model coincides with the 2021 one on all the instances; in the other 23 the median over the instances of the ratio between the iterations of MiniSat and those of the model (the measure (6) of section 6) changes by at most 0.03 (on `flat30-60` at cap 500, e.g., the medians are 1.833 for Graph-Q-SAT and 2.000 for GAT-Q-SAT on both stacks). We have not traced the residual differences, which may be due to the non-determinism of the scatter operations on a GPU; they are in any case much smaller than the differences discussed in section 6. Indeed, inference exercises the graph construction, the message passing, the decoder and the stepping of the environment together, and hence this is a reasonably stringent test of all but the training path. The latter was checked by retraining the two coloring models from scratch on the new stack (these are the runs of 2026 of section 6).

Finally, the evaluation code implements the three restricted heuristics of [29], which reduce the number of forward passes. In the first one the model takes the first T decisions and VSIDS the rest. In the second one a forward pass gives the L best actions, which are executed in order (skipping those whose variable has meanwhile been assigned) before the network is queried again. In the third one the Q-values computed at the root initialize the activities of VSIDS,

which then runs unaided (this required a new method of the solver, which writes the activities and rebuilds the order heap). We do not report results for them, since on the instances of section 6 the search is too short for the saving in inference to show.

6 Computational results

We now compare the two models on test instances that are increasingly far from the training ones. As in [15], the quality of a heuristic is measured by the Median Relative Iteration Reduction

$$\text{MRIR} = \text{median}\ \frac{\#\text{iterations of MiniSat}}{\#\text{iterations of the model}}, \tag{6}$$

where the median is over the instances of a family, and an iteration is a branching decision (of the model or of VSIDS). Each evaluation is run at a given cap (cf. section 2), after which VSIDS takes the remaining decisions; unless stated otherwise the cap is 500, as in training.

Reference solver. The model-guided solver runs without restarts, while MiniSat can be run with or without them (with the restart sequence of [19]), and the choice of the denominator of (6) is by no means inconsequential. On graph coloring MiniSat with restarts takes up to 26% fewer iterations than without (on `flat175-417`); on `uf250-1065` and `uuf250-1065`, however, it takes 2.2 and 2.9 times more. We divide by the stronger of the two on each family, i.e., by MiniSat with restarts on graph coloring (as in [15]) and without restarts on uniform random 3-SAT. The one exception is `uuf100-430`, where the run with restarts takes 6% fewer iterations but where our logs (those of 2021, the only ones for the random families) record only the other one; hence, the MRIR reported there is slightly favorable to the models. On the families of section 6.4 we also divide by the stronger of the two.

Runs. Each run is trained once on one family, with the configuration of section 2, and is evaluated at its last checkpoint. On graph coloring, Graph-Q-SAT and GAT-Q-SAT have three runs each trained on `flat50-115` (i.e., three seeds): one trained in 2021 on the original stack and two in 2026 on the ported one, the last with a fixed seed of the random number generators (the others were not seeded). The two controls of section 4 have two runs each, trained in 2026 with fixed seeds. On uniform random 3-SAT each variant has two runs trained on satisfiable formulas (one on `uf50-218`, one on `uf100-430`), whose results we average, and two trained on the unsatisfiable `uuf50-218` and `uuf100-430`, used only for a comparison in section 6.5. For a group of runs we report the mean of the MRIR of its runs.

Instances and machine. Each test set has 100 instances (101 for `flat150-360`): the held-out test part for the families used in training, and the whole family for the others (cf. the appendix). All the evaluations of 2026 were run on a Colab instance with a GPU; the MRIR, a ratio of iteration counts, does not depend on the machine, and the running times are discussed in section 6.6.

6.1 Graph coloring

Table 1 reports the MRIR of the coloring-trained models on the eight graph-coloring families. Column "seeds" gives the MRIR of the run of 2021 and of the two of 2026, "mean" their mean, and "Δ" the difference of the means (positive when GAT-Q-SAT takes fewer decisions). Since the families range from 30 to 200 nodes, the largest test instances are four times the size of the training ones.

GAT-Q-SAT takes fewer decisions than Graph-Q-SAT on all eight families. Up to 100 nodes, which include the test part of the training family, the difference is between 0.07 and 0.58 and it does not grow with the size (it is 0.58 at 75 nodes and 0.12 at 100). From 125 nodes on it is between 0.66 and 1.66. This is mostly (but not only) due to Graph-Q-SAT, whose MRIR decreases from 2.64 at 100 nodes to 1.05 at 200 (i.e., almost to parity with the reference solver),

	Graph-Q-SAT		GAT-Q-SAT		
family	seeds	mean	seeds	mean	Δ
`flat30-60`	1.83 / – / –	1.83	2.00 / – / –	2.00	+0.17
`flat50-115`	2.18 / – / –	2.18	2.25 / – / –	2.25	+0.07
`flat75-180`	2.37 / – / –	2.37	2.94 / – / –	2.94	+0.58
`flat100-239`	2.64 / – / –	2.64	2.76 / – / –	2.76	+0.12
`flat125-301`	2.43 / – / –	2.43	3.09 / – / –	3.09	+0.66
`flat150-360`	1.74 / – / –	1.74	3.40 / – / –	3.40	+1.66
`flat175-417`	1.17 / – / –	1.17	2.59 / – / –	2.59	+1.42
`flat200-479`	1.05 / – / –	1.05	2.64 / – / –	2.64	+1.59

Table 1: Coloring-trained models on graph coloring: MRIR against MiniSat with restarts at cap 500, per seed (2021 / 2026 / 2026 with a fixed seed) and mean over the seeds.

while that of GAT-Q-SAT stays between 2.6 and 3.4. In other words, the two models are quite close on instances of about the training size, and GAT-Q-SAT degrades much less when the instances grow. With the other two seeds the pattern is the same on all the families, although the size of the difference changes from seed to seed by up to X.

Figure 3 shows the MRIR of the runs of 2021 as a function of the cap, family by family, on a common vertical scale (the runs of 2026 were evaluated only at the caps of Tables 1 and 4). On most families Graph-Q-SAT reaches its best MRIR at a cap of 50. It then degrades as it is allowed more decisions, down to 0.77 on `flat200-479` at cap 1000; the MRIR of GAT-Q-SAT does not decrease with the cap on any family. On the larger instances, then, the decisions that Graph-Q-SAT takes after the first few tens make the search longer, and the ranking of Table 1 does not seem to depend on the choice of the cap (at least above a few tens of decisions).

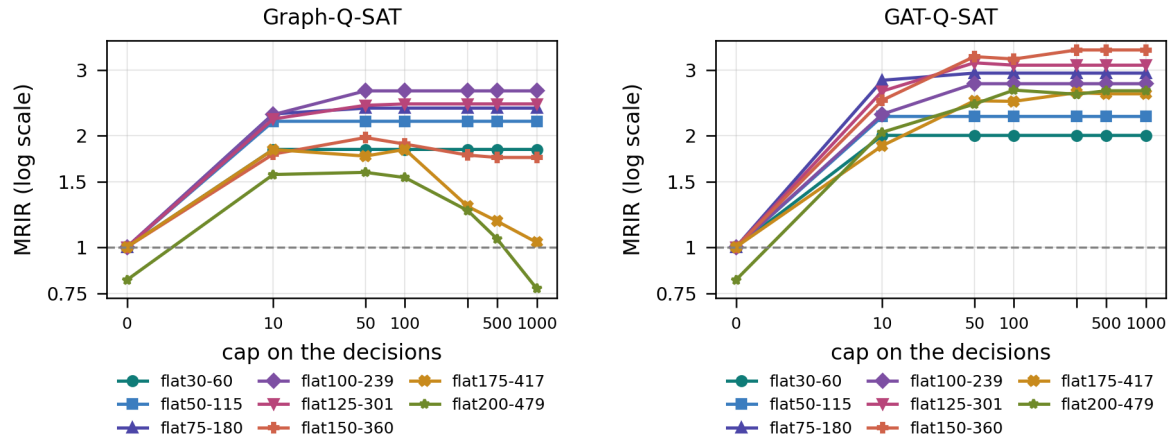


Figure 3: Coloring-trained runs of 2021: MRIR against MiniSat with restarts as a function of the cap, on the eight graph-coloring families, for Graph-Q-SAT (left) and GAT-Q-SAT (right), on a common log scale. The point at cap 0 is MiniSat without restarts, i.e., the solver that the models drive, left alone.

6.2 Capacity and aggregation

We now check whether the advantage of GAT-Q-SAT on graph coloring is due to the attention or to its larger network. Table 2 reports, on the same families and at the same cap as Table 1,

the mean MRIR of the four coloring-trained variants: Graph-Q-SAT, Graph-Q-SAT wide (with the capacity of GAT-Q-SAT), attention aggregation (5) (with the capacity of Graph-Q-SAT), and GAT-Q-SAT (cf. section 4); the two controls are the mean of two seeds, the other two of three.

family	Graph-Q-SAT	Graph-Q-SAT wide	attention aggregation	GAT-Q-SAT
`flat30-60`	–	–	–	–
`flat50-115`	–	–	–	–
`flat75-180`	–	–	–	–
`flat100-239`	–	–	–	–
`flat125-301`	–	–	–	–
`flat150-360`	–	–	–	–
`flat175-417`	–	–	–	–
`flat200-479`	–	–	–	–
parameters	71144	113504	71627	113256

Table 2: Coloring-trained variants on graph coloring: MRIR against MiniSat with restarts at cap 500, mean over the seeds (three for Graph-Q-SAT and GAT-Q-SAT, two for the controls).

[To be written from Table 2: whether Graph-Q-SAT wide closes the gap (in which case the advantage is one of capacity), whether the attention aggregation reaches GAT-Q-SAT at a sixth of its added parameters, and on which families; the range of the differences and their size against the variation across the seeds.]

6.3 A change of the training distribution

We now evaluate on graph coloring the models trained on satisfiable uniform random 3-SAT, whose formulas have none of the structure of a coloring problem. On five of the eight families Graph-Q-SAT is below the reference solver, its MRIR ranging from 0.63 to 1.16 (0.89 on average). GAT-Q-SAT, instead, is above it on seven (from 0.94 to 1.75, 1.44 on average); the difference between the two is 0.44 to 0.97 on the first seven families. On `flat200-479`, however, it drops to 0.10, and GAT-Q-SAT too falls slightly below parity. Figure 4 compares at cap 500 the three regimes that have been measured on the same families, and Figure 5 shows the models trained on random formulas as a function of the number of variables of the test instances.
Thus, GAT-Q-SAT is the better of the two at all sizes on graph coloring (although by only 0.10 on `flat200-479`), and the worse one at all sizes on satisfiable random formulas (cf. section 6.5). The ranking may therefore depend on the test family more than on the particular instances, and the learned weighting of the neighbors seems to be of some use even on a structure that the model has not seen in training. We remark, however, that the absolute difference is smaller here than in section 6.1 (0.55 against 0.78 on average), and that the MRIR of both models decreases with the size of the coloring instances.

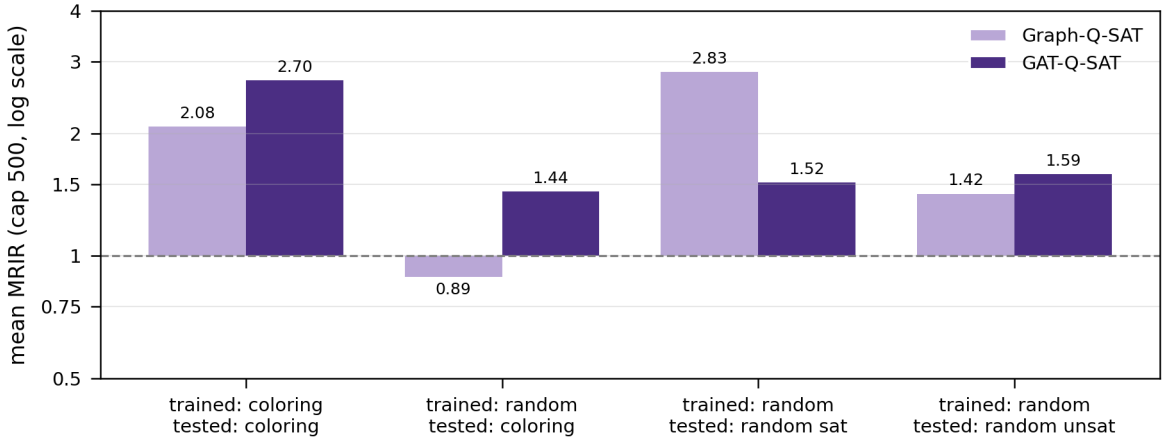


Figure 4: MRIR at cap 500, mean over the families: trained and tested on graph coloring (eight families), trained on satisfiable random 3-SAT and tested on graph coloring (eight families), and trained on satisfiable random 3-SAT and tested on the satisfiable and on the unsatisfiable families of Table 3; log scale, bars start at parity.

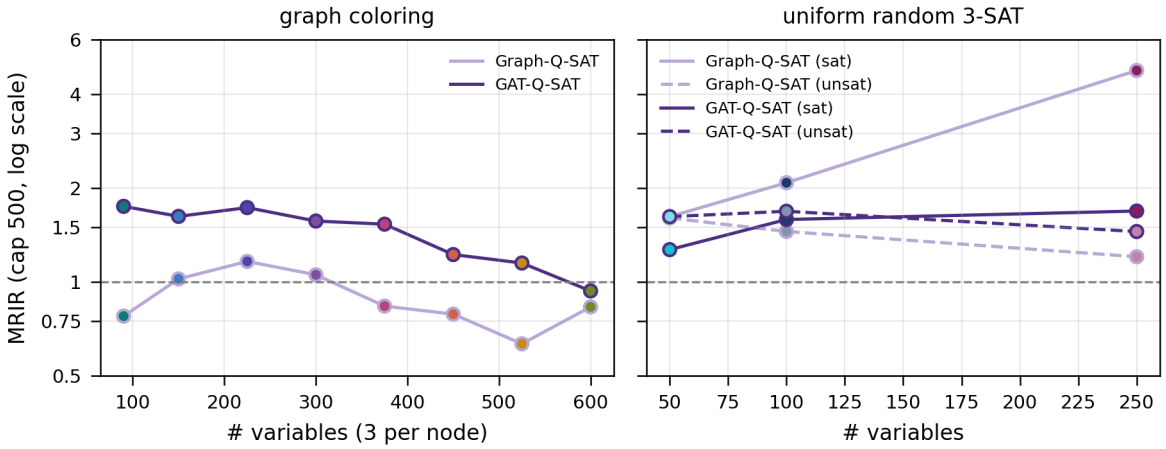


Figure 5: Models trained on satisfiable random 3-SAT: MRIR at cap 500 as a function of the number of variables of the test instances, on graph coloring (left, three variables per node) and on random 3-SAT (right, unsatisfiable families dashed), on a log scale.

6.4 Cross-domain transfer

All the families above are random, either in the formula or in the graph, and therefore they are of limited help to a user of a learned heuristic, who would like to know whether a heuristic trained on the instances at hand is of any use on different ones. We therefore evaluated the coloring-trained models (all their seeds, with no retraining) on the structured families of SATLIB. We took all of them, with the exception of those whose instances exceed 2000 variables or 150000 clauses (the size of the largest instances that we could evaluate on our hardware in reasonable time), and an instance enters the panel only if MiniSat, with and without restarts, solves it within 10 minutes (cf. the appendix). The resulting panel has thirteen families and N instances: AIM [1], `jnh`, `dubois`, `pret`, pigeon hole, parity, all interval series, inductive inference, circuit fault analysis, the Beijing set, towers of Hanoi, planning [13] and quasigroup completion [10]. Several of these families are rather "small" (from 2 to 50 instances), and on some of them MiniSat needs very few decisions after its preprocessing (fewer than 10 on 46% of the AIM instances, e.g.); we report the MRIR on all the instances, as in [15], and we also check that the ranking does not change when those instances are left out. We evaluate at a cap of 200 to contain the cost of the largest instances, and divide by the stronger MiniSat run on each family.

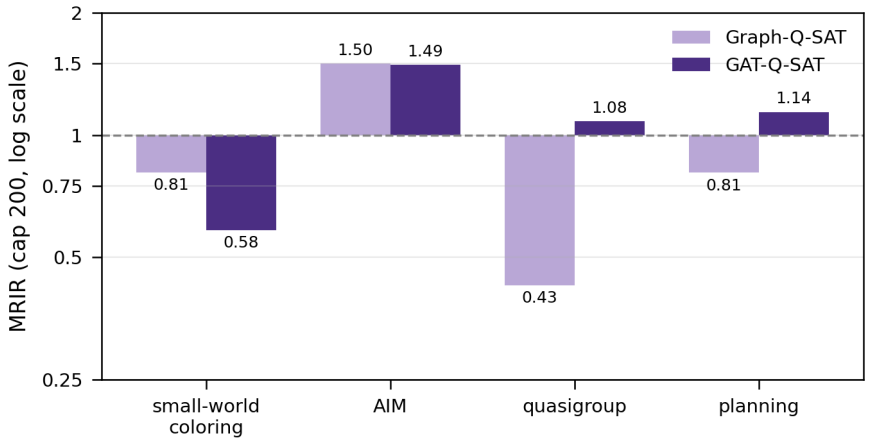


Figure 6: Coloring-trained models on thirteen SATLIB families not seen in training (number of instances in parentheses): MRIR at cap 200 against the stronger MiniSat run on each family, mean over the seeds, on a log scale; bars start at parity.

[To be written from Figure 6 and from the summary table: on how many families each variant is above the reference solver and above the other variants, by how much (as a range), the families where the ranking flips, the effect of leaving out the instances solved in fewer than 10 decisions, and the variation across the seeds; the families with fewer than 10 instances are commented as indications only.]

Small-world coloring [32] allows a more controlled experiment, since it varies the structure of the graph within one problem. Its instances are the 5-coloring problems of graphs on 100 vertices obtained from a ring lattice by rewiring each edge with probability p, and SATLIB provides nine levels, from $p = 1$ (a random graph) to $p = 2^{-8}$ (almost the lattice), plus the lattice itself. Figure 7 reports the MRIR of the coloring-trained models on 20 instances per level (one for the lattice). [To be written from Figure 7: whether the MRIR of the two models, and their difference, change with p, i.e., with the distance of the graphs from the random ones of the training family.]

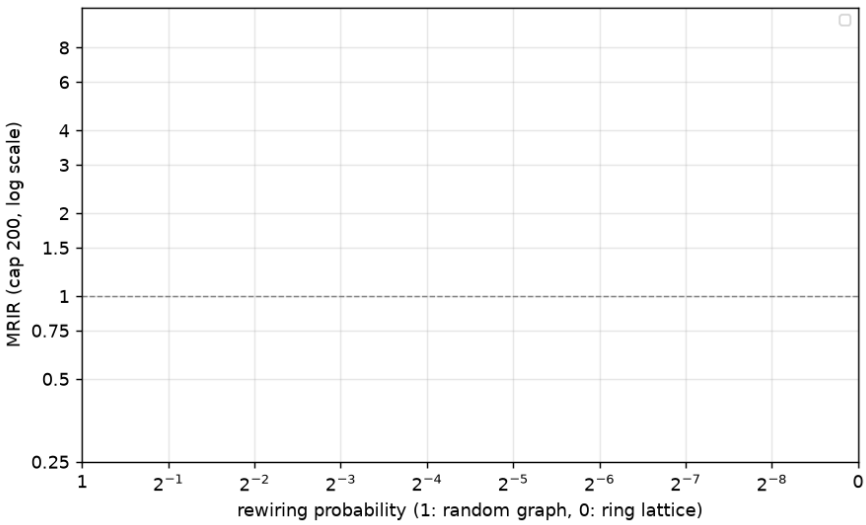


Figure 7: Coloring-trained models on small-world coloring: MRIR at cap 200 against the stronger MiniSat run, as a function of the rewiring probability of the graphs (log scale; 20 instances per level, one for the lattice).

6.5 Uniform random 3-SAT

On uniform random 3-SAT the ranking of the two models is reversed. Table 3 reports the MRIR of the models trained on satisfiable formulas (mean of the runs trained on `uf50-218` and `uf100-430`) on the satisfiable and on the unsatisfiable families of the same generator. Figure 8 shows the same models as a function of the cap. On the satisfiable families Graph-Q-SAT is better at all sizes: by 0.35 at 50 variables, by 0.49 at 100, and by 3.09 at 250, where its MRIR is 4.78 against 1.69. (The run of Graph-Q-SAT trained on `uf50-218` alone reaches 6.19 there, on instances five times larger than the training ones.) This is consistent with the remark of section 4: at the phase transition the neighbors of a node are statistically alike, and the attention coefficients have little to learn.

	satisfiable			unsatisfiable		
	`uf50-218`	`uf100-430`	`uf250-1065`	`uuf50-218`	`uuf100-430`	`uuf250-1065`
Graph-Q-SAT	1.62	2.08	4.78	1.61	1.45	1.21
GAT-Q-SAT	1.27	1.59	1.69	1.62	1.69	1.45

Table 3: Models trained on satisfiable random 3-SAT (mean of the runs trained on `uf50-218` and `uf100-430`): MRIR against MiniSat without restarts at cap 500.

However, the advantage of Graph-Q-SAT does not carry over to the unsatisfiable formulas. On these the two models are level at 50 variables (1.61 and 1.62), and GAT-Q-SAT is ahead at 100 (1.69 against 1.45) and at 250 (1.45 against 1.21). Two explanations are possible, and our measurements do not separate them: either Graph-Q-SAT has learned something about where the satisfying assignments of these formulas tend to be, which is of no use when there are none, or proving unsatisfiability leaves little room to the branching heuristic, and the two models end up somewhat closer. The runs trained on unsatisfiable formulas improve Graph-Q-SAT (from 1.61 to 1.96 on `uuf50-218` and from 1.45 to 1.56 on `uuf100-430`) but not GAT-Q-SAT (1.63 and 1.50), which is consistent with the first explanation without excluding the second. All in all, the regime in which the sum of Graph-Q-SAT is clearly superior is that of satisfiable uniform random formulas, which is also the one with the least structure for a weighting of the

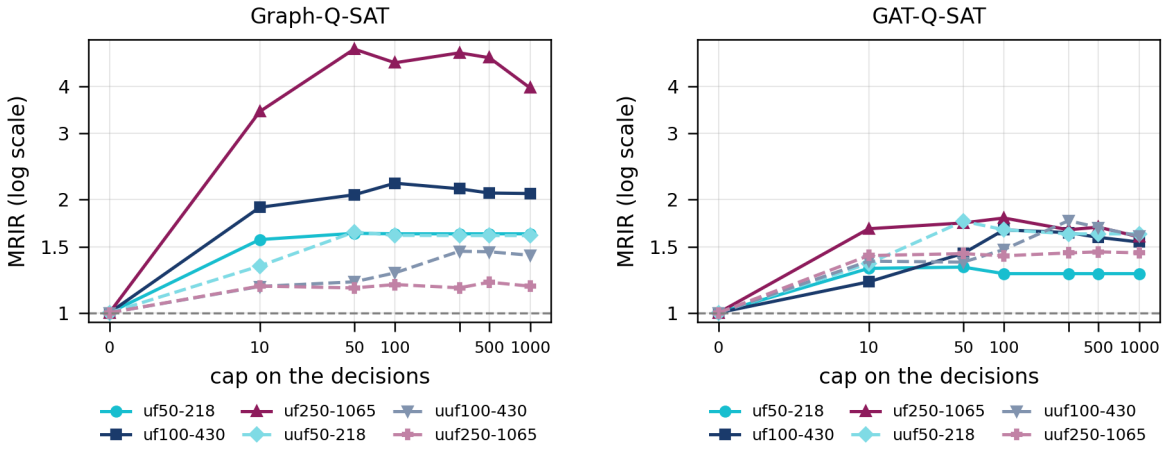


Figure 8: Models trained on satisfiable random 3-SAT (mean of the two runs): MRIR against MiniSat without restarts as a function of the cap, on the satisfiable (`uf`) and unsatisfiable (`uuf`) families, for Graph-Q-SAT (left) and GAT-Q-SAT (right), on a common log scale; the unsatisfiable families are dashed.

neighbors to exploit.

6.6 Iterations and running time

The MRIR counts iterations, which are the quantity on which the heuristic acts. However, a decision of the model costs a forward pass of the network on the current graph, and the attention layers make that pass more expensive. We measure the time per instance around the episode. It includes the parsing of the formula, the variable elimination, the construction of the graph at each decision, the forward passes and the part of the search left to VSIDS (but not the loading of the model and the runs of MiniSat that give the denominator of (6), which are computed once per family and stored with it). We report medians over the instances, since a few instances usually take much longer than the others. The load of the Colab instance was not under our control; repeating two of the evaluations gave medians within $X\%$ of the first ones, and hence we only comment on differences much larger than that.

	cap 50		cap 500		cap 1000	
	sec	MRIR	sec	MRIR	sec	MRIR
Graph-Q-SAT	2.55	1.59	26.51	1.05	57.67	0.77
GAT-Q-SAT	3.52	2.43	12.20	2.64	12.19	2.64

Table 4: Coloring-trained models on `flat200-479`: median seconds per instance ("sec") and MRIR against MiniSat with restarts, as a function of the cap (mean over the seeds).

Table 4 reports time and MRIR on the largest coloring family at three values of the cap. At cap 50 a decision of GAT-Q-SAT is more expensive than one of Graph-Q-SAT (3.52 against 2.55 seconds for the same number of model decisions), as expected. At larger caps, instead, GAT-Q-SAT is the faster of the two (by a factor of 2 at cap 500 and of almost 5 at cap 1000), since it terminates in fewer decisions; indeed, each decision of a model costs far more than the VSIDS decisions it replaces. Graph-Q-SAT pays 22 times the time of cap 50 to reach a lower MRIR, and it ends up below the reference solver at cap 1000. Going from cap 50 to cap 500 improves GAT-Q-SAT by 0.21, and going to 1000 changes nothing, since its episodes terminate before the cap is reached.

In absolute terms both models take seconds per instance on formulas with at most a few thousand clauses, which MiniSat solves in a small fraction of a second. At this scale neither model is competitive with the solver in running time, and the reduction of the iterations measures the quality of the decisions (their convenience being a different matter). This is the motivation of the restricted heuristics of [29] (cf. section 5) and of the offline inference of [34], whose benefit may only show on instances where the search is long enough to dominate the cost of inference.

7 Conclusions

We have added two edge-featured graph attention layers to the core block of Graph-Q-SAT, and compared the resulting model, GAT-Q-SAT, with Graph-Q-SAT at matched training set and configuration. On all the graph-coloring families GAT-Q-SAT took fewer decisions. The difference was small on instances of about the training size and large (up to about 1.7) on those three and four times larger, where Graph-Q-SAT approached the reference solver. [One sentence on the controls.] GAT-Q-SAT was also the better of the two when trained on random formulas and tested on graph coloring and, with the caveats due to the size of the families, on some of the structured families that neither model had seen. On satisfiable uniform random 3-SAT, instead, Graph-Q-SAT was clearly superior, consistently with the absence of structure for the attention to weight. On the unsatisfiable formulas of the same generator, however, the two were level or GAT-Q-SAT was ahead. Within the limits of these experiments, the attention layers seem to make the learned heuristic less sensitive to the size and to the nature of the test

instances when these have some structure, at the cost of about 60% more parameters and of a more expensive decision.

Several limitations should be kept in mind. First, three training seeds per variant (two for the controls) are enough for the larger differences but not for those of a few tenths of MRIR, which may well be within the variance of DQN training. [Second, the conclusion drawn from the controls, with its limits.] On the instances used here neither model is competitive with MiniSat in running time, and the restricted heuristics that should reduce the cost of inference are implemented but have not been measured on instances large enough for them to matter. Finally, several families of the transfer panel have fewer than 10 instances, and the largest instances of SATLIB (e.g., those of bounded model checking) are beyond the size that we could evaluate, which is perhaps the most serious limit of the panel; a panel of larger industrial instances, e.g., that of [17], would be the natural next test, and we hope to address it in the future.

Acknowledgements

The implementation that we started from is the one released by Kurin et al. with [15], and the solver underneath it is MiniSat [6].

References

[1] Yuichi Asahiro, Kazuo Iwama, and Eiji Miyano. Random generation of test instances with controlled attributes. In *Cliques, Coloring, and Satisfiability*, volume 26 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, pages 377–393. American Mathematical Society, 1996.

[2] Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, et al. Relational inductive biases, deep learning, and graph networks. *arXiv:1806.01261*, 2018.

[3] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: A methodological tour d'horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.

[4] Shaked Brody, Uri Alon, and Eran Yahav. How attentive are graph attention networks? In *International Conference on Learning Representations*, 2022.

[5] Joseph C. Culberson and Feng Luo. Exploring the k-colorable landscape with iterated greedy. In *Cliques, Coloring, and Satisfiability*, volume 26 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, pages 245–284. American Mathematical Society, 1996.

[6] Niklas Eén and Niklas Sörensson. An extensible SAT-solver. In *Theory and Applications of Satisfiability Testing (SAT 2003)*, volume 2919 of *Lecture Notes in Computer Science*, pages 502–518. Springer, 2004.

[7] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[8] Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems 32*, 2019.

[9] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1263–1272, 2017.

[10] Carla P. Gomes and Bart Selman. Problem structure in the presence of perturbations. In *Proceedings of the Fourteenth National Conference on Artificial Intelligence (AAAI)*, pages 221–226, 1997.

[11] Wenxuan Guo, Hui-Ling Zhen, Xijun Li, Wanqian Luo, Mingxuan Yuan, Yaohui Jin, and Junchi Yan. Machine learning methods in solving the Boolean satisfiability problem. *Machine Intelligence Research*, 20(5):640–655, 2023.

[12] Holger H. Hoos and Thomas Stützle. SATLIB: An online resource for research on SAT. In Ian P. Gent, Hans van Maaren, and Toby Walsh, editors, *SAT2000: Highlights of Satisfiability Research in the Year 2000*, volume 63 of *Frontiers in Artificial Intelligence and Applications*, pages 283–292. IOS Press, 2000.

[13] Henry Kautz and Bart Selman. Pushing the envelope: Planning, propositional logic, and stochastic search. In *Proceedings of the Thirteenth National Conference on Artificial Intelligence (AAAI)*, pages 1194–1201, 1996.

[14] Elias B. Khalil, Pierre Le Bodic, Le Song, George L. Nemhauser, and Bistra Dilkina. Learning to branch in mixed integer programming. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, 2016.

[15] Vitaly Kurin, Saad Godil, Shimon Whiteson, and Bryan Catanzaro. Can Q-learning with graph networks learn a generalizable branching heuristic for a SAT solver? In *Advances in Neural Information Processing Systems 33*, pages 9608–9621, 2020.

[16] Michail G. Lagoudakis and Michael L. Littman. Learning to select branching rules in the DPLL procedure for satisfiability. *Electronic Notes in Discrete Mathematics*, 9:344–359, 2001.

[17] Zhaoyu Li, Jinpei Guo, and Xujie Si. G4SATBench: Benchmarking and advancing SAT solving with graph neural networks. *Transactions on Machine Learning Research*, 2024.

[18] Jia Hui Liang, Vijay Ganesh, Pascal Poupart, and Krzysztof Czarnecki. Learning rate based branching heuristic for SAT solvers. In *Theory and Applications of Satisfiability Testing (SAT 2016)*, volume 9710 of *Lecture Notes in Computer Science*, pages 123–140. Springer, 2016.

[19] Michael Luby, Alistair Sinclair, and David Zuckerman. Optimal speedup of Las Vegas algorithms. *Information Processing Letters*, 47(4):173–180, 1993.

[20] João Marques-Silva, Inês Lynce, and Sharad Malik. Conflict-driven clause learning SAT solvers. In *Handbook of Satisfiability*, volume 336 of *Frontiers in Artificial Intelligence and Applications*, chapter 4, pages 133–182. IOS Press, second edition, 2021.

[21] João P. Marques-Silva and Karem A. Sakallah. GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521, 1999.

[22] David G. Mitchell, Bart Selman, and Hector J. Levesque. Hard and easy distributions of SAT problems. In *Proceedings of the Tenth National Conference on Artificial Intelligence (AAAI)*, pages 459–465, 1992.

[23] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[24] David Mojžíšek, Jan Hula, Ziwei Li, Ziyu Zhou, and Mikoláš Janota. Neural approaches to SAT solving: Design choices and interpretability. *International Journal of Approximate Reasoning*, 190:109609, 2026.

[25] Matthew W. Moskewicz, Conor F. Madigan, Ying Zhao, Lintao Zhang, and Sharad Malik. Chaff: Engineering an efficient SAT solver. In *Proceedings of the 38th Design Automation Conference (DAC)*, pages 530–535, 2001.

[26] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations*, 2016.

[27] Daniel Selsam and Nikolaj Bjørner. Guiding high-performance SAT solvers with unsat-core predictions. In *Theory and Applications of Satisfiability Testing (SAT 2019)*, volume 11628 of *Lecture Notes in Computer Science*, pages 336–353. Springer, 2019.

[28] Daniel Selsam, Matthew Lamm, Benedikt Bünz, Percy Liang, Leonardo de Moura, and David L. Dill. Learning a SAT solver from single-bit supervision. In *International Conference on Learning Representations*, 2019.

[29] Mikhail Shirokikh, Ilya Shenbin, Anton Alekseev, and Sergey Nikolenko. Machine learning for SAT: Restricted heuristics and new graph representations. *arXiv:2307.09141*, 2023.

[30] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, 2016.

[31] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.

[32] Toby Walsh. Search in a small world. In *Proceedings of the Sixteenth International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1172–1176, 1999.

[33] Fei Wang and Tiark Rompf. From gameplay to symbolic reasoning: Learning SAT solver heuristics in the style of Alpha(Go) Zero. *arXiv:1802.05340*, 2018.

[34] Wenxi Wang, Yang Hu, Mohit Tiwari, Sarfraz Khurshid, Kenneth L. McMillan, and Risto Miikkulainen. NeuroBack: Improving CDCL SAT solving using graph neural networks. In *International Conference on Learning Representations*, 2024.

[35] Emre Yolcu and Barnabás Póczos. Learning local search heuristics for Boolean satisfiability. In *Advances in Neural Information Processing Systems 32*, 2019.

[36] Wenjie Zhang, Zeyu Sun, Qihao Zhu, Ge Li, Shaowei Cai, Yingfei Xiong, and Lu Zhang. NLocalSAT: Boosting local search with solution prediction. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1177–1183, 2020.

A The instances, and how the numbers are regenerated

All the instances come from SATLIB [12], in DIMACS form, and are stored in the repository of the code (cf. section 1). The families used in training (`flat50-115`, `uf50-218`, `uf100-430`,

`uuf50-218` and `uuf100-430`) are split once, with a fixed seed, into training, validation and test parts of 80%, 10% and 10% (800, 100 and 100 formulas for most of them). The evaluations of section 6 use the test parts of these families and the whole of the others, i.e., `flat30-60`, `flat75-180`, `flat100-239`, `flat125-301`, `flat150-360`, `flat175-417` and `flat200-479` for graph coloring, and `uf250-1065` and `uuf250-1065` for random 3-SAT, with 100 formulas each (101 for `flat150-360`).

The panel of section 6.4 contains all the structured SATLIB families with at most 2000 variables and 150000 clauses per instance, namely AIM (72 instances with 50, 100 and 200 variables, 48 satisfiable with a single solution and 24 unsatisfiable), `jnh` (50, random with clauses of variable length), `dubois` (13, unsatisfiable), `pret` (8, unsatisfiable 2-coloring), pigeon hole (5, unsatisfiable), parity learning (25: `par8`, `par16` and the compressed `par32`), all interval series (4), inductive inference (41), circuit fault analysis (9: the `bf` and `ssa` instances within the limit), the Beijing set (8), towers of Hanoi (2), planning (8, blocks world and logistics) and quasigroup completion (20). The rule leaves out all of `bmc` and of the large graph coloring set `gcp`, the uncompressed `par32`, the largest blocks-world, logistics, quasigroup and circuit instances and most of the Beijing set; the large random formulas of `LRAN` are not structured, and are not used. The small-world set contains 20 instances for each of the nine SATLIB levels `sw100-8-lp0-c5` to `sw100-8-lp8-c5` (drawn with a fixed seed from the 100 of each level; level k has rewiring probability 2^{-k}) and the single lattice instance `sw100-8-p0-c5`, all with 500 variables and 3100 clauses. None of these instances is used in training.

For each family the iterations of MiniSat with and without restarts are computed once, instance by instance, and stored with the family, so that the denominator of (6) is the same for all the models; for the families of section 6.4 this is done by `make_metadata.py`, which runs the two MiniSat configurations on each instance with a limit of 10 minutes and leaves out (listing it) any instance that is not solved within the limit (X instances). The evaluation of a model on a family at a cap writes one line per instance, with the time, the iterations of the model and the two iterations of MiniSat. The logs of 2021, which are the only ones for the random families, record only the ratio against MiniSat without restarts. The tables of section 6 are printed by `paper_numbers.py`, which reads the logs, groups the runs by variant and training family, and applies the choice of the denominator described at the beginning of section 6. The figures are drawn from the same logs, and with the same grouping, by `make_plots.py` (Figures 3 and 8), `paper_analysis.py` (Figures 4 and 5) and `transfer_study.py` (Figure 6), which also runs the evaluations of section 6.4; those of the other sections are run by `reeval.sh`. The logs written in 2026 are kept with the checkpoints, outside the repository.